



ЗВІТ

Лабораторна робота № 8

Дисципліна: Базы даних та інформаційні системи

Тема: Розширені можливості Redis

Виконав

Студент 3 курсу

Групи МІТ-31

Факультету інформаційних технологій

Бабяк Володимир

1. Вступ

Мета роботи: закріпити знання про роботу з Redis та ознайомитися з розширеним функціоналом — транзакціями, скриптами на Lua, публікацією/підпискою (Pub/Sub) та потоками Redis Streams.

У даній лабораторній роботі було виконано:

- Транзакції в Redis: команди MULTI, EXEC, DISCARD, WATCH
- Lua-скрипти: атомарне виконання логіки через команду EVAL
- Механізм Pub/Sub: публікація та підписка на повідомлення в реальному часі
- Redis Streams: додавання та читання поточкових подій (XADD, XRANGE, XREAD)
- Python-скрипт redis_advanced.py з реалізацією всіх чотирьох механізмів

2. Практична частина

Крок 1 — Транзакції (MULTI / EXEC / WATCH)

Транзакція дозволяє виконати кілька команд Redis атомарно — або всі, або жодна. Команди між MULTI і EXEC ставляться в чергу зі статусом QUEUED.

Проста транзакція:

```
127.0.0.1:6379> -- Проста транзакція
(error) ERR unknown command '--', with args beginning with: 'Проста' 'транзакція'
127.0.0.1:6379> MULTI
OK
127.0.0.1:6379(TX)> SET user:1 "Володимир"
QUEUED
127.0.0.1:6379(TX)> SET user:2 "Олена"
QUEUED
127.0.0.1:6379(TX)> INCR visits
QUEUED
127.0.0.1:6379(TX)> EXEC
1) OK
2) OK
3) (integer) 1
127.0.0.1:6379>
```

Команда	Результат
MULTI	OK
SET user:1 "Володимир"	QUEUED
SET user:2 "Олена"	QUEUED
INCR visits	QUEUED
EXEC	1) OK 2) OK 3) (integer) 1

Транзакція з WATCH для імітації конкурентного доступу:

```
127.0.0.1:6379> -- WATCH для конкурентного доступу
(error) ERR unknown command '--', with args beginning with: 'WATCH' 'для' 'конкурентного' 'доступу'
127.0.0.1:6379> SET balance 100
OK
127.0.0.1:6379> WATCH balance
OK
127.0.0.1:6379> MULTI
OK
127.0.0.1:6379(TX)> DECR balance
QUEUED
127.0.0.1:6379(TX)> INCR purchases
QUEUED
127.0.0.1:6379(TX)> EXEC
1) (integer) 99
2) (integer) 1
127.0.0.1:6379>
```

Команда	Результат
SET balance 100	OK
WATCH balance	OK
MULTI	OK
DECR balance	QUEUED
INCR purchases	QUEUED
EXEC	1) (integer) 99 2) (integer) 1

WATCH встановлює спостереження за ключем. Якщо ключ змінюється іншим клієнтом до EXEC — транзакція скасовується і EXEC повертає nil. У даному випадку balance успішно зменшився до 99, purchases збільшився до 1.

Крок 2 — Lua-скрипти (EVAL)

Lua-скрипти виконуються атомарно на стороні Redis-сервера. Це дозволяє реалізувати складну логіку без race condition.

Виконаний скрипт перевіряє наявність ключа і створює його тільки якщо він не існує:

```
127.0.0.1:6379> EVAL "if redis.call('exists', KEYS[1]) == 0 then return redis.call('set', KEYS[1], ARGV[1]) else return 'exists' end" 1 testkey
"hello"
"exists"
```

Виклик	Результат
1-й виклик (ключ не існує)	ОК — ключ створено
2-й виклик (ключ існує)	"exists" — ключ вже є

Перший виклик повернув ОК — ключ `testkey` було створено зі значенням `'hello'`. Другий виклик повернув `'exists'`, оскільки ключ вже існував. Логіка виконана атомарно без проміжних станів.

Крок 3 — Pub/Sub (SUBSCRIBE / PUBLISH)

Для демонстрації Pub/Sub було відкрито два термінали у VS Code (Split Terminal).

Термінал 1 — підписка на канал:

SUBSCRIBE news

Термінал 2 — публікація повідомлень:

PUBLISH news "Redis is awesome!"

PUBLISH news "Hello from Volodymyr!"

```

PS C:\ADB Lab> docker exec -it redis-lab7 redis-cli
127.0.0.1:6379> SUBSCRIBE news
1) "subscribe"
2) "news"
3) (integer) 1
Reading messages... (press Ctrl-C to quit or any key to type com
Reading messages... (press Ctrl-C to quit or any key to type com
Reading messages... (press Ctrl-C to quit or any key to type com
1) "message"
2) "news"
3) "Redis is awesome!"
Reading messages... (press Ctrl-C to quit or any key to type com
Reading messages... (press Ctrl-C to quit or any key to type com
Reading messages... (press Ctrl-C to quit or any key to type com
Reading messages... (press Ctrl-C to quit or any key to type com
1) "message"
2) "news"
3) "Hello from Volodymyr!"
Reading messages... (press Ctrl-C to quit or any key to type com
^C
quit)

```

```
PS C:\DB Lab> docker exec -it redis-lab7 redis-cli
127.0.0.1:6379> PUBLISH news "Redis is awesome!"
(integer) 5
127.0.0.1:6379> PUBLISH news "Hello from Volodymyr!"
(integer) 5
127.0.0.1:6379>
```

Дія	Результат
SUBSCRIBE news	1) "subscribe" 2) "news" 3) (integer) 1
PUBLISH "Redis is awesome!"	(integer) 5 — кількість підписників
PUBLISH "Hello from Volodymyr!"	(integer) 5
Термінал 1 отримав	"Redis is awesome!" та "Hello from Volodymyr!"

Термінал 1 автоматично отримав обидва повідомлення у форматі:

- 1) "message"
- 2) "news"
- 3) "Redis is awesome!"

Pub/Sub працює за принципом «вогонь і забув» (fire-and-forget) — повідомлення не зберігаються, якщо підписник не підключений в момент публікації.

Крок 4 — Redis Streams (XADD / XRANGE / XREAD)

Redis Streams — це структура даних для збереження послідовності подій з унікальними ID на основі мітки часу.

```
127.0.0.1:6379> XADD mystream * sensor-id 1234 temperature 19.8
"1781176328786-0"
127.0.0.1:6379> XADD mystream * sensor-id 1234 temperature 22.1
"1781176328790-0"
127.0.0.1:6379> XADD mystream * sensor-id 5678 temperature 18.5
"1781176328796-0"
127.0.0.1:6379>
127.0.0.1:6379> XRANGE mystream - +
1) 1) "1781176313318-0"
   2) 1) "sensor-id"
      2) "1234"
      3) "temperature"
      4) "19.8"
2) 1) "1781176313326-0"
   2) 1) "sensor-id"
      2) "1234"
      3) "temperature"
      4) "22.1"
3) 1) "1781176313330-0"
   2) 1) "sensor-id"
      2) "5678"
      3) "temperature"
      4) "18.5"
4) 1) "1781176328786-0"
   2) 1) "sensor-id"
      2) "1234"
      3) "temperature"
      4) "19.8"
5) 1) "1781176328790-0"
   2) 1) "sensor-id"
      2) "1234"
      3) "temperature"
      4) "22.1"
6) 1) "1781176328796-0"
   2) 1) "sensor-id"
      2) "5678"
      3) "temperature"
      4) "18.5"
```

```
127.0.0.1:6379> XREAD COUNT 2 STREAMS mystream 0
1) 1) "mystream"
   2) 1) 1) "1781176313318-0"
      2) 1) "sensor-id"
         2) "1234"
         3) "temperature"
         4) "19.8"
      2) 1) "1781176313326-0"
         2) 1) "sensor-id"
            2) "1234"
            3) "temperature"
            4) "22.1"
127.0.0.1:6379> █
```

Додавання подій до потоку:

Команда	Результат (ID події)
XADD sensor 1234, temp 19.8	"1781176328786-0"
XADD sensor 1234, temp 22.1	"1781176328790-0"
XADD sensor 5678, temp 18.5	"1781176328796-0"

Читання всіх подій:

Повернуло всі записи з потоку у хронологічному порядку. Кожна подія має унікальний ID формату timestamp-sequence та поля sensor-id і temperature.

Читання перших 2 подій через XREAD:

Подія	Дані
1781176313318-0	sensor-id: 1234, temperature: 19.8
1781176313326-0	sensor-id: 1234, temperature: 22.1

XREAD з параметром COUNT 2 повернув тільки перші 2 події починаючи від ID 0. Це корисно для посторінкового читання великих потоків.

Крок 5 — Python-скрипт `redis_advanced.py`

Написано скрипт, що реалізує всі чотири механізми через бібліотеку `redis-py`:


```

1  import redis
2  import threading
3  import time
4
5  r = redis.Redis(host='localhost', port=6379, decode_responses=True)
6
7  # =====
8  # 1. ТРАНЗАКЦІЇ
9  # =====
10 print("=== 1. Транзакція ===")
11 pipe = r.pipeline()
12 pipe.multi()
13 pipe.set("user:1", "Володимир")
14 pipe.set("user:2", "Олена")
15 pipe.incr("visits")
16 results = pipe.execute()
17 print(f"Результати транзакції: {results}")
18
19 # =====
20 # 2. LUA-СКРИПТ
21 # =====
22 print("\n=== 2. Lua-скрипт ===")
23 lua_script = """
24 if redis.call('exists', KEYS[1]) == 0 then
25     return redis.call('set', KEYS[1], ARGV[1])
26 else
27     return 'exists'
28 end
29 """
30 r.delete("luakey")
31 result1 = r.eval(lua_script, 1, "luakey", "hello")
32 print(f"Перший виклик: {result1}")
33 result2 = r.eval(lua_script, 1, "luakey", "hello")
34 print(f"Другий виклик: {result2}")
35
36 # =====
37 # 3. PUB/SUB
38 # =====
39 print("\n=== 3. Pub/Sub ===")
40
41 def subscriber():
42     r_sub = redis.Redis(host='localhost', port=6379, decode_responses=True)
43     pubsub = r_sub.pubsub()
44     pubsub.subscribe("news")
45     print("Підписник: очікую повідомлення...")
46     count = 0

```

```

41 def subscriber():
42     r_sub = redis.Redis(host='localhost', port=6379, decode_responses=True)
43     pubsub = r_sub.pubsub()
44     pubsub.subscribe("news")
45     print("Підписник: очікую повідомлення...")
46     count = 0
47     for message in pubsub.listen():
48         if message['type'] == 'message':
49             print(f"Отримано: {message['data']}")
50             count += 1
51             if count >= 2:
52                 break
53     pubsub.unsubscribe()
54
55 thread = threading.Thread(target=subscriber)
56 thread.start()
57 time.sleep(0.5)
58
59 r.publish("news", "Redis is awesome!")
60 r.publish("news", "Привіт від Володимира!")
61
62 thread.join()
63
64 # =====
65 # 4. REDIS STREAMS
66 # =====
67 print("\n=== 4. Redis Streams ===")
68 r.delete("mystream")
69
70 r.xadd("mystream", {"sensor-id": "1234", "temperature": "19.8"})
71 r.xadd("mystream", {"sensor-id": "1234", "temperature": "22.1"})
72 r.xadd("mystream", {"sensor-id": "5678", "temperature": "18.5"})
73
74 events = r.xrange("mystream", "-", "+")
75 print("Всі події:")
76 for event_id, data in events:
77     print(f" ID: {event_id} | Дані: {data}")
78
79 read = r.xread({"mystream": "0"}, count=2)
80 print(f"\nXREAD (перші 2): {read[0][1]}")

```

Результати виконання:

```
PS C:\DB Lab> & "C:/Program Files/Python313/python.exe" "c:/DB Lab/lab 8/redis_advanced.py"
=== 1. Транзакція ===
Результати транзакції: [True, True, 2]

=== 2. Lua-скрипт ===
Перший виклик: OK
Другий виклик: exists

=== 3. Pub/Sub ===
Підписник: очікую повідомлення...
Отримано: Redis is awesome!
Отримано: Привіт від Володимира!

=== 4. Redis Streams ===
Всі події:
ID: 1781176423849-0 | Дані: {'sensor-id': '1234', 'temperature': '19.8'}
ID: 1781176423850-0 | Дані: {'sensor-id': '1234', 'temperature': '22.1'}
ID: 1781176423852-0 | Дані: {'sensor-id': '5678', 'temperature': '18.5'}

XREAD (перші 2): [('1781176423849-0', {'sensor-id': '1234', 'temperature': '19.8'}), ('1781176423850-0', {'sensor-id': '1234', 'temperature': '22.1'})]
PS C:\DB Lab>
```

Секція	Результат
1. Транзакція	Результати транзакції: [True, True, 2]
2. Lua-скрипт	Перший виклик: OK Другий виклик: exists
3. Pub/Sub	Отримано: Redis is awesome! / Привіт від Володимира!
4. Redis Streams	3 події додано, XREAD повернув перші 2

Скрипт успішно виконався без помилок. Pub/Sub реалізований через threading — підписник запущений в окремому потоці, основний потік публікує повідомлення.

3. Теоретичні висновки

Використання розширених механізмів Redis у розподілених системах

Механізм	Де використовувати	Приклад
Транзакції	Фінансові операції, резервування	Списання балансу + запис покупки атомарно
Lua-скрипти	Складна атомарна логіка	Rate limiting, умовне оновлення кешу
Pub/Sub	Реальний час, нотифікації	Чат, live-оновлення дашборду

Streams	IoT, логування, аналітика	Збір даних з сенсорів температури
---------	---------------------------	-----------------------------------

Плюси і мінуси Redis як брокера повідомлень

Переваги:

- Надвисока швидкість — всі операції виконуються в пам'яті, затримка мікросекунди
- Простота налаштування — мінімальна конфігурація порівняно з RabbitMQ або Kafka
- Streams зберігають історію подій на відміну від Pub/Sub
- Підтримка Consumer Groups — кілька споживачів можуть читати один потік

Недоліки:

- Pub/Sub не зберігає повідомлення — якщо підписник офлайн, повідомлення втрачається
- Обмежена пам'ять — всі дані в RAM, великі об'єми дорого коштують
- Відсутність складної маршрутизації як у RabbitMQ (exchanges, bindings)
- Менш надійний для критичних черг порівняно з Kafka або RabbitMQ

4. Відповіді на запитання для самоперевірки

1. Як працюють транзакції в Redis і для чого потрібна WATCH?

Транзакції в Redis реалізовані через блок MULTI/EXEC. Після MULTI всі команди ставляться в чергу зі статусом QUEUED і виконуються атомарно при EXEC. DISCARD скасовує транзакцію.

WATCH встановлює спостереження за ключами. Якщо хоч один з ключів змінився між WATCH і EXEC іншим клієнтом — EXEC повертає nil і транзакція не виконується. Це механізм оптимістичного блокування для захисту від race condition.

2. Для чого використовують Lua-скрипти в Redis?

Lua-скрипти виконуються атомарно на сервері Redis без переривань. Це дозволяє реалізувати складну умовну логіку (перевірити — оновити) як єдину операцію. Використовується для gate

limiting, умовного оновлення кешу, складних лічильників та уникнення race condition між кількома командами.

3. У чому суть моделі Pub/Sub і які її обмеження?

Pub/Sub — модель асинхронного обміну повідомленнями. Видавець (publisher) публікує повідомлення в канал, не знаючи про підписників. Підписники (subscribers) отримують всі повідомлення каналу в реальному часі.

Обмеження: повідомлення не зберігаються — якщо підписник не підключений в момент публікації, він їх втрачає. Немає підтвердження доставки. Не підходить для надійних черг завдань.

4. Що таке Redis Streams? Як зчитуються події?

Redis Streams — лог-структура для зберігання послідовності подій. Кожна подія має унікальний ID (timestamp-sequence) та набір полів. На відміну від Pub/Sub, події зберігаються постійно.

Читання: XRANGE повертає події в діапазоні ID (- означає початок, + кінець). XREAD читає події починаючи від заданого ID з можливістю обмеження кількості через COUNT. Consumer Groups дозволяють кільком споживачам читати потік з підтвердженням обробки.

5. Коли Redis може замінити RabbitMQ?

- Прості черги без складної маршрутизації — Redis Streams справляються
- Коли потрібна максимальна швидкість і невеликий розмір повідомлень
- Real-time нотифікації та live-оновлення через Pub/Sub
- Коли Redis вже використовується в проекті як кеш — немає сенсу додавати окремий брокер
- Для IoT та логування подій де важлива хронологія — Redis Streams з Consumer Groups

5. Висновок

У ході виконання лабораторної роботи було успішно досліджено розширені можливості Redis: транзакції, Lua-скрипти, Pub/Sub та Streams.

Транзакції з MULTI/EXEC/WATCH забезпечують атомарність операцій та захист від конкурентного доступу. Lua-скрипти дозволяють виконувати складну логіку атомарно на сервері. Pub/Sub

підходить для real-time нотифікацій, а Streams — для надійного збереження та обробки поточкових даних.

Написано Python-скрипт `redis_advanced.py`, який демонструє всі чотири механізми програмно. Результати підтвердили коректну роботу кожного з механізмів відповідно до документації Redis.